

Tower of Babel

Laporan Teknis Babel Gateway

Seleksi Sister 2026 Part 2 Nomor 2

Daftar Isi

1. Menyalakan environment dan melihat isinya
 2. Berkenalan dengan Service A
 3. Berkenalan dengan Service B, dan sebuah kejutan
 4. Berkenalan dengan Service C
 5. Membaca ledger: temuan yang mengubah desain
 6. Arsitektur gateway
 7. Service registry
 8. Strategi translasi protokol
 9. Strategi routing
 10. Model konkurensi
 11. Penanganan kegagalan
 12. Bonus 1: migrasi protokol saat runtime dan kompatibilitas
 13. Bonus 2: pemuatan adapter saat runtime
 14. Bonus 3: integrasi lanjutan
 15. Bonus 4: penanganan di luar spesifikasi
 16. Keputusan teknis dan kompromi
 17. Keterbatasan implementasi
 18. Verifikasi
-

1. Menyalakan environment dan melihat isinya

Sebelum menulis satu baris kode pun, hal pertama yang saya lakukan adalah menyalakan environment yang sudah disediakan dan melihat apa yang sebenarnya ada di dalamnya. Folder `towerofbabel` berisi empat image Docker, dokumentasi protokol, dan sebuah control plane untuk menginjeksi fault.

```
cd towerofbabel
docker compose up -d
docker compose ps
```

Hasilnya empat container berjalan: `service-a` di port 8101, `service-b` di 8201, `service-c` di 8301 (UDP), dan `control-plane` di 8090.

Dokumentasi di `protocols/` menjelaskan bahwa ketiganya berbicara dengan dialek yang sangat berbeda:

Service	Transport	Framing	Korelasi	Integritas
<code>service-a</code>	TCP / HTTP	HTTP dan JSON biasa	header <code>X-Request-ID</code>	dijamin TCP
<code>service-b</code>	TCP	header biner 16 byte lalu payload JSON	<code>requestId</code> di header frame dan di payload	dijamin TCP
<code>service-c</code>	UDP	header biner 20 byte, JSON, lalu CRC32	<code>requestId</code> dan <code>sequence</code>	CRC32

Dokumentasi itu bagus, tetapi dokumentasi selalu bercerita tentang bagaimana sesuatu seharusnya bekerja. Yang saya butuhkan adalah bagaimana ketiga service ini benar-benar bekerja, termasuk hal-hal yang tidak tertulis. Jadi saya menulis sebuah klien Python kecil untuk mengetuk ketiganya secara langsung, mengirim request yang benar maupun yang sengaja salah, lalu merekam apa yang keluar.

Keputusan untuk melakukan probing ini ternyata sangat menentukan. Beberapa temuan di bawah membalik pilihan desain yang tampak paling masuk akal di atas kertas.

2. Berkenalan dengan Service A

Service A adalah yang paling ramah. HTTP dan JSON, tidak ada yang aneh.

```
curl -X POST http://localhost:8101/v1/execute \
  -H 'Content-Type: application/json' \
  -H 'X-Protocol-Version: 1' \
  -H 'X-Request-ID: probe-1' \
  -d '{"operation_name":"echo","operation_arguments":{"value":"babel"}}'
```

```
{"operation_name":"echo","operation_result":{"value":"babel"},
  "request_id":"probe-1","service_name":"service-a"}
```

Endpoint `/v1/capabilities` memberi tahu bahwa Service A hanya bisa `echo`, `uppercase`, dan `metadata`. Mencoba `sum` di sini menghasilkan HTTP 400 dengan kode `OPERATION_NOT_SUPPORTED`.

Satu hal yang saya catat: operasi `metadata` mengembalikan objek metadata langsung di dalam `operation_result`, tanpa dibungkus `value`. Artinya aturan normalisasi hasil tidak bisa seragam untuk semua operasi. Ini akan jadi penting nanti.

3. Berkenalan dengan Service B, dan sebuah kejutan

Service B jauh lebih menarik. Saya menulis encoder frame di Python, lalu mengirim satu frame `ECHO` :

```
babe 01 00 0000002a 1122334455667788 {"args":{"value":"babe1"},"opCode":"ECHO"}
```

Responsnya kembali dengan `resultData.value` berisi `"babe1"`. Sejauh ini sesuai dokumen.

Lalu saya mencoba sesuatu yang tidak diminta dokumen: mengirim tujuh frame sekaligus dalam satu koneksi tanpa menunggu balasan satu per satu. Hasilnya mengejutkan.

```
frame terkirim : 1 2 3 4 5 6 7
respons kembali: 2 7 6 4 5 3 1
```

Service B ternyata benar-benar melakukan multiplexing. Ia menjawab dalam urutan apa pun yang ia mau. Kalau klien saya mengasumsikan respons datang berurutan, saya akan memasangkan jawaban ke request yang salah, dan yang lebih buruk lagi, saya baru akan menyadarinya ketika beban tinggi.

Temuan ini langsung mengubah rancangan klien TCP saya. Alih-alih satu koneksi per request, saya membangunnya dengan satu goroutine pembaca dan sebuah peta `pending` yang diindeks `requestID`, persis seperti klien protokol multiplexed sungguhan.

Kejutan kedua muncul ketika saya melakukan `ECHO` terhadap sebuah angka:

```
{"errorData":null,"requestId":1234605616436508552,
"resultData":{"numericResult":42},"serviceId":"service-b"}
```

Perhatikan kuncinya. `echo` sebuah string kembali di `resultData.value`, tetapi `echo` sebuah angka kembali di `resultData.numericResult`. Kunci jawaban bergantung pada tipe argumen, bukan pada nama operasi. Adapter yang hanya membaca `value` akan benar untuk string dan diam-diam salah untuk angka.

Kejutan ketiga muncul saat saya sengaja mengirim frame dengan magic yang salah:

```
{"errorData":{"code":"INVALID_FRAME","message":"Frame header is invalid."},
"requestId":0,"resultData":null,"serviceId":"service-b"}
```

`requestId` nya nol. Artinya error frame semacam ini tidak bisa dikorelasikan ke request mana pun. Gateway harus memperlakukannya sebagai frame liar, bukan sebagai jawaban untuk siapa pun.

4. Berkenalan dengan Service C

Service C adalah yang paling rapuh, karena UDP tidak menjanjikan apa pun. Saya membangun paketnya di Python, lengkap dengan CRC32 di ekor:

```
c0de 01 01 00000007 1122334455667788 01 00 0011 {"value":"babel"} 2e32d28d
```

Responsnya kembali dengan `sequence` dan `requestId` yang sama, tipe pesan 2, dan `result.value` berisi `"babel"`. Bagus.

Lalu saya mengirim paket request yang sama dua kali. Service C menjawab dua kali, dengan isi yang identik. Ia tidak melakukan deduplikasi sama sekali. Artinya deduplikasi harus dikerjakan sepenuhnya oleh gateway, di pasangan `(requestID, seq)`.

Saya juga merusak CRC-nya dengan sengaja. Responsnya adalah paket error yang, sama seperti Service B, membawa `requestId` nol dan `seq` nol. Tidak bisa dikorelasikan.

5. Membaca ledger: temuan yang mengubah desain

Control plane menyediakan sebuah ledger di `/ledger` yang mencatat setiap panggilan ke backend. Setiap entri punya field `stage` yang bernilai `received` atau `executed`. Ini ternyata adalah alat paling berharga di seluruh environment, karena ia memberi tahu bukan hanya bahwa sebuah request sampai, tetapi apakah operasinya benar-benar dijalankan.

Spesifikasi soal mengatakan gateway boleh melakukan fallback "asalkan tidak menciptakan operasi duplikat". Pertanyaannya: dari mana gateway tahu sebuah percobaan yang gagal sudah terlanjur dijalankan atau belum? Ledger inilah jawabannya, dan saya memakainya untuk memetakan setiap mode fault satu per satu.

Metodenya sederhana. Aktifkan satu skenario, kirim satu request, tunggu, lalu baca ledger.

```
curl -X POST http://localhost:8090/reset -H "X-Babel-Control-Token: babel-local-dev"
curl -X POST http://localhost:8090/scenario/service-a-slow -H "X-Babel-Control-Token: babel-local-dev"
# kirim satu request, tunggu 3 detik
curl -s http://localhost:8090/ledger -H "X-Babel-Control-Token: babel-local-dev"
```

Untuk `service-a-slow`, yang menginjeksi fault `delayed_response`, hasilnya:

```
received service-a uppercase fault_applied=delayed_response t=280.07
executed service-a uppercase t=282.59
```

Delay diterapkan lebih dulu, lalu operasinya tetap dijalankan. Ini penting sekali. Kalau gateway saya timeout pada 2000 ms lalu berpindah ke Service B, akan ada dua entri `executed` untuk satu request logis. Itu persis "operasi duplikat" yang dilarang. Dan ternyata memang ada skenario bernama `fallback-unsafe` yang menginjeksi fault ini.

Sekarang bandingkan dengan `fallback-safe`, yang menginjeksi `connection_termination`:

```
received service-a uppercase fault_applied=connection_termination
```

Hanya `received`. Tidak ada `executed`. Operasinya tidak pernah dijalankan, jadi memindahkannya ke backend lain benar-benar aman.

Saya melanjutkan pemetaan ini untuk semua mode fault yang bisa saya picu:

Fault	Mencapai tahap <code>executed</code> ?	Kesimpulan
<code>delayed_response</code>	ya	tidak aman untuk fallback
<code>connection_termination</code>	tidak	aman
<code>http_503</code>	tidak	aman
<code>invalid_json</code>	tidak	aman
<code>missing_required_field</code>	tidak	aman
<code>unsupported_version</code>	ya, tetapi responsnya valid	aman untuk dialihkan

Inilah pondasi seluruh aturan fallback di gateway saya. Bukan tebakan, bukan intuisi, tetapi hasil pengukuran.

Satu temuan lagi dari fase ini, yang tidak kalah penting. Saya penasaran apakah endpoint health Service A juga terkena fault. Ternyata tidak:

```
GET /v1/health -> 200 dalam 0.21 detik  
POST /v1/execute -> 200 dalam 2.67 detik
```

Fault hanya berlaku pada jalur `execute`. Health tetap sehat. Gateway yang mempercayai probe health untuk mengambil keputusan routing akan salah persis di situasi ini. Jadi saya memutuskan sejak awal: probe hanya sinyal liveness, kualitas routing datang dari hasil request yang benar-benar teramati.

6. Arsitektur gateway

Setelah tahu apa yang saya hadapi, barulah saya merancang gateway-nya. Bahasanya Go 1.25, tanpa dependensi eksternal sama sekali. Alasannya praktis: protokol biner butuh kontrol byte yang eksplisit, tiga model transport yang berbeda butuh primitif konkurensi yang ringan, dan build tanpa dependensi berarti `docker compose up` tidak bergantung pada registry mana pun yang harus bisa dihubungi saat penilaian.

Alur sebuah request dari atas ke bawah:

```

POST /execute
|
+- 1. validasi envelope      pelanggaran kontrak caller -> HTTP 400
+- 2. validasi argumen       jawabannya sama apa pun backend-nya
+- 3. admission control      batas konkurensi global dan per-backend
|
+- 4. ROUTER
|   lookup capability di registry persisten
|   urutan kandidat deterministik: priority, lalu service_id
|   deadline per percobaan diiris dari budget caller
|   circuit breaker per (service, versi protokol)
|   gerbang fallback: hanya bila percobaan gagal terbukti tidak bekerja
|
+- 5. ADAPTER  satu per service dan versi, deklaratif, bisa diganti runtime
|   request  : operasi logis dan argumen -> dialek backend
|   response : validasi dulu, baru normalisasi ke envelope
|
+- 6. TRANSPORT
|   http-json      HTTP dengan connection pool
|   tcp-frame-json frame multiplexed, korelasi lewat request id
|   udp-crc-json   ARQ, adaptive RTO, dedupe, reassembly, window

```

Pemisahan lapisan ini ketat dan disengaja. Ada satu aturan yang saya pegang sepanjang implementasi:

Tidak ada satu pun kode di atas lapisan adapter yang boleh mengenal `operation_result`, `resultData`, `numericResult`, `errorData`, atau `serviceId`.

Dialek backend berhenti di adapter. Akibatnya lapisan HTTP publik tidak perlu tahu apakah sebuah kegagalan berasal dari magic frame yang salah atau dari Service A yang menjawab 429. Semuanya sudah dinormalisasi menjadi satu taksonomi error sebelum sampai ke sana.

Pembagian pakatnya:

Paket	Tanggung jawab
apimodel	Tipe wire API publik, validasi envelope dan argumen
gwerr	Taksonomi error ternormalisasi berikut semantik retry-nya
registry	Registry persisten: service, capability, variant, health
adapter	Spec deklaratif, manager hot-swap, tiga implementasi family
transport/tcpmux	Klien frame multiplexed
transport/udprel	Lapisan reliabilitas di atas datagram
router	Routing, budget waktu, retry, fallback, breaker
breaker	Circuit breaker per (service, versi)
health	Probe periodik dan pelaporan kesehatan
api	Endpoint publik dan endpoint operator
obs	Structured logging dan counter
idgen	Alokator correlation id yang aman melintasi restart

7. Service registry

Registry adalah satu-satunya sumber kebenaran struktural. Ia menyimpan service apa yang ada, protokol apa yang dipakai, di mana endpoint-nya, operasi apa yang boleh dilayani, versi protokol mana yang terpasang, dan seberapa sehat backend saat ini.

Isinya kira-kira seperti ini:

```
{
  "schema_version": 1,
  "revision": 7,
  "id_high_water": 1872746255968371200,
  "services": {
    "service-b": {
      "service_id": "service-b",
      "protocol": "tcp-frame-json",
      "endpoint": "service-b:8201",
      "enabled": true,
      "operations": [
        {"name": "echo", "priority": 20, "replay_safe": false},
        {"name": "sum", "priority": 10, "replay_safe": false},
        {"name": "reverse", "priority": 10, "replay_safe": false}
      ],
      "variants": [
        {"version": 1, "weight": 100, "adapter_name": "tcp-frame-json-v1"},
        {"version": 2, "weight": 0, "adapter_name": "tcp-frame-json-v2"}
      ]
    }
  },
  "adapter_specs": { }
}
```

Ada dua properti yang menanggung beban paling besar di sini.

Semua yang struktural dipersistensi. Perubahan runtime oleh operator seperti menonaktifkan backend, menggeser bobot versi, atau memuat adapter baru, semuanya bertahan melewati restart. Ini persis yang diminta requirement ketahanan konfigurasi.

Memori dan disk tidak pernah berbeda. Setiap mutasi menempuh satu jalur yang sama: salin, terapkan, validasi seluruh dokumen, tulis ke disk, baru publikasikan. Kalau penulisannya gagal, konfigurasi yang sedang hidup tidak berubah sama sekali. Dengan begitu gateway tidak pernah bisa berada dalam keadaan menjalankan konfigurasi yang tidak akan bisa dipulihkannya setelah restart.

Validasi berjalan atas seluruh dokumen, bukan hanya entri yang disentuh, supaya satu edit yang buruk tidak bisa meninggalkan registry yang inkonsisten secara global.

Yang sengaja tidak dipersistensi

Health adalah observasi runtime yang berubah setiap interval probe. Kalau saya menuliskannya setiap kali probe selesai, sinyal observability berubah menjadi trafik disk. Nilai terakhir tetap dilipat ke snapshot secara oportunistik, supaya gateway yang baru restart bisa melaporkan sesuatu yang lebih berguna daripada `unknown` sebelum probe pertamanya selesai. Tetapi nilai itu tidak pernah dipercaya untuk keputusan routing.

Penulisan yang atomik

Urutannya: tulis ke file sementara, `fsync`, `rename` menimpa target, lalu `fsync` direktori. Operasi `rename` dalam satu direktori bersifat atomik, jadi pembaca (atau gateway yang crash) hanya bisa melihat dokumen lama yang utuh atau dokumen baru yang utuh. Tidak pernah registry yang tersobek separuh.

Dokumen rusak dikarantina, bukan ditimpa

Kalau registry tidak bisa dibaca, gateway memindahkannya ke `registry.json.corrupt-<timestamp>`, boot dengan default, dan memunculkan peringatan di `/status`. Menempanya diam-diam justru akan menghancurkan state operator yang requirement-nya meminta dilestarikan.

Schema version yang tidak dikenal ditolak keras dengan alasan yang sama. Dokumen dari build masa depan bisa saja mengikat adapter yang build ini tidak mampu penuhi, dan menebak isinya lebih berbahaya daripada menolaknya.

8. Strategi translasi protokol

Adapter adalah dokumen, bukan kode

Ini keputusan desain yang paling saya senangi. Satu `Spec` mendeskripsikan satu versi protokol: endpoint, parameter wire, pemetaan field per operasi, dan sekumpulan golden test vector. Tiga adapter bawaan gateway ini adalah spec, dan mereka dimuat lewat jalur yang persis sama dengan apa pun yang dikirim ke `/admin/adapters`.

Alasannya bukan kosmetik. Kalau jalur deklaratif tidak sanggup mengekspresikan tiga backend nyata, maka fitur "pemuatan adapter saat runtime" hanyalah pajangan yang menempel di samping mesin yang sebenarnya. Karena hanya ada satu jalur, jalur itu tidak bisa membusuk tanpa ketahuan.

Contoh spec untuk Service B:

```
{
  "name": "tcp-frame-json-v1",
  "family": "tcp-frame-json",
  "service_id": "service-b",
  "version": 1,
  "wire": {
    "magic_hex": "babe", "max_payload": 65536,
    "operation_field": "opCode", "arguments_field": "args",
    "correlation_field": "requestId",
    "result_field": "resultData", "error_field": "errorData",
    "error_code_field": "code", "error_message_field": "message",
    "error_retryable_field": "retryable"
  },
  "operations": {
    "sum": {
      "wire": "SUM",
      "arg_map": {"values": "numberList"},
      "result_keys": ["numericResult", "value"]
    },
    "metadata": {"wire": "METADATA", "passthrough": true}
  },
  "self_test": [ "...vektor yang direkam dari backend..." ]
}
```

Apa saja yang diterjemahkan

Serialisasi dan framing. Tiap family punya codec sendiri. Body JSON untuk HTTP, header 16 byte big-endian ditambah payload untuk frame, header 20 byte ditambah payload ditambah CRC32 untuk datagram.

Nama operasi. `uppercase` menjadi `uppercase` di Service A, `UPPERCASE` di Service B, dan di Service C bukan string sama sekali melainkan satu byte opcode di dalam header.

Nama argumen. Identitas, kecuali `sum` di Service B yang mengubah `values` menjadi `numberList`.

Bentuk hasil. Ini bagian yang paling halus, dan bagian yang paling diuntungkan oleh probing di awal. Aturannya: kalau operasi ditandai `passthrough`, objek hasil diteruskan apa adanya (itulah `metadata`). Kalau tidak, kunci pertama dari `result_keys` yang keberadaannya terdeteksi dipakai lalu dibungkus ulang menjadi `{"value": ...}`.

Pengecekkannya berbasis keberadaan kunci, bukan nilai yang bukan null, supaya `{"value": null}` yang sah tidak salah dianggap tidak ada.

Dan `result_keys` untuk Service B berisi dua entri, `["value", "numericResult"]`, karena probing di bagian 3 menunjukkan kunci jawaban bergantung pada tipe argumen. Tanpa probing itu saya hampir pasti hanya menulis `["value"]` dan menghasilkan bug yang hanya muncul untuk argumen numerik.

Error. Kode error backend diteruskan apa adanya, tidak dipetakan ulang. Backend tahu lebih banyak tentang alasan penolakannya daripada substitusi apa pun dari sisi gateway, dan kontrak Gateway API hanya mewajibkan bentuk envelope-nya, bukan kosakata kodenya.

Versi. Divalidasi pada respons, bukan sekadar diklaim pada request. Untuk frame dan datagram, versinya ada di header. Untuk HTTP, header `X-Protocol-Version` pada respons diperiksa kalau ada. Pemeriksaan inilah yang menangkap skenario `unsupported-version`, di mana Service A menjawab dengan `X-Protocol-Version: 9` sementara body-nya sendiri sepenuhnya valid. Tanpa pemeriksaan ini gateway akan meneruskan jawaban dari protokol yang tidak ia pahami.

Golden vector, dan kenapa bukan skema saja

Setiap adapter membawa vektor uji berisi byte request dan response yang direkam dari backend sungguhan menggunakan klien Python terpisah. Vektor negatifnya diturunkan dari rekaman yang sama, dengan merusak tepat satu field.

Sebelum sebuah adapter boleh melayani trafik, ia harus lulus tiga hal:

1. mereproduksi byte request yang direkam, persis sama;
2. mendekode respons yang direkam menjadi hasil ternormalisasi yang benar;
3. menolak rekaman yang dirusak, dengan kode error yang tepat.

Kenapa tidak cukup validasi skema? Karena skema tidak menangkap kelas kesalahan yang paling mungkin terjadi pada spec yang ditulis manusia. File `demo/adapters/broken-adapter.json` memetakan argumen `sum` ke `numbers`, bukan `numberList`. Secara struktur file itu sempurna. Yang menangkapnya adalah

ketidakmampuannya menghasilkan byte yang pernah saya lihat dari backend:

```
want babe01000000002e...7b226e756d6265724c697374223a5b312c322c335d7d...
got  babe01000000002b...7b226e756d62657273223a5b312c322c335d7d...
```

Mekanisme ini terbukti berguna. Selama pengembangan, ia menangkap satu bug nyata yang saya jelaskan di bagian 15.

9. Strategi routing

Berbasis capability, dibaca dari registry

Kandidat sebuah operasi adalah service yang `enabled`, mendeklarasikan operasi tersebut, dan punya minimal satu variant yang membawa bobot. Urutannya total dan deterministik: `priority` per operasi lebih dulu, lalu `service_id` sebagai pemecah seri.

Operasi	service-a	service-b	service-c
<code>echo</code>	10	20	30
<code>uppercase</code>	10	20	tidak bisa
<code>sum</code>	tidak bisa	10	20
<code>reverse</code>	tidak bisa	10	tidak bisa
<code>metadata</code>	10	20	30

Priority default mencerminkan keandalan transport: HTTP di atas frame TCP, frame TCP di atas datagram UDP. Semuanya data, jadi operator bisa mengubah urutan routing tanpa rebuild.

Determinisme di sini bukan soal estetika. Fallback yang tidak deterministik tidak bisa dipikirkan ulang saat insiden dan tidak bisa direproduksi saat penilaian.

`preferred_service`

Kalau preferensi menunjuk backend yang mampu, ia ditaruh di depan, dan sisa urutan deterministiknya tetap menjadi rantai fallback.

Kalau preferensi menunjuk backend yang tidak mampu melayani operasi itu, kontrak mengizinkan dua pembacaan: menolak, atau memilih alternatif yang aman. Default gateway ini adalah melayani caller. Request-nya well-formed, ada backend yang mampu, dan menggagalkannya adalah jawaban yang lebih buruk daripada menghormati maksud panggilan. Keputusan ini dicatat di log setiap kali terjadi, dan `BABEL_PREFERRED_INCAPABLE=strict` memilih pembacaan yang satunya.

Operasi tidak dikenal versus tidak ada rute

Dua hal ini dibedakan, karena artinya berbeda bagi caller:

- operasi tidak ada di registry mana pun menghasilkan `UNSUPPORTED_OPERATION` dengan `retryable: false`, disertai daftar operasi yang memang ada;
- operasi ada tetapi tidak ada backend aktif yang bisa melayaninya menghasilkan `NO_ROUTE_AVAILABLE` dengan `retryable: true`, karena itu keadaan konfigurasi, bukan kesalahan klien.

Keduanya menjawab dengan `service_id: null`, sesuai kontrak.

Validasi argumen sebelum dispatch

`sum` dengan nilai bukan angka, `uppercase` dengan angka, nilai di luar rentang 32-bit bertanda, semuanya ditolak sebelum backend mana pun disentuh. Ada dua alasan. Pertama, jawabannya jadi deterministik terlepas dari backend mana yang akan dipilih routing. Kedua, panggilan yang salah bentuk tidak menghabiskan satu percobaan backend beserta irisan budget waktu caller.

10. Model konkurensi

Bentuk dasarnya

Satu goroutine per request HTTP yang masuk, mengikuti model `net/http`. Di dalamnya, percobaan ke backend berjalan berurutan, tidak pernah paralel. Ini yang menjamin invarian paling penting di seluruh sistem:

Tepat satu envelope per request klien. Dua backend tidak akan pernah sama-sama menjawab panggilan yang sama.

Hedging, yaitu mengirim ke dua backend sekaligus lalu mengambil yang tercepat, akan melanggar invarian ini sekaligus melanggar larangan operasi duplikat. Jadi tidak saya implementasikan, meskipun ia akan memperbaiki latensi ekor.

Korelasi

Setiap percobaan ke backend mendapat identifier `uint64` yang dialokasikan gateway dan tidak pernah dipakai ulang. Strukturnya: 20 bit rendah untuk counter dalam proses, sisanya milidetik Unix.

Saat start, alokator disemai pada `max(now << 20, persisted_high_water + gap)`. Dengan begitu identifier tidak berulang bahkan ketika:

- proses restart dalam milidetik yang sama, karena high water yang dipersistensi mendominasi;
- jam sistem mundur, karena high water yang dipersistensi tetap mendominasi;
- file state hilang, karena rantai dari jam tetap maju.

`request_id` milik klien tidak pernah dipakai sebagai correlation id backend. Klien boleh mengulanginya, dan identifier yang berulang akan membuat respons basi dari request yang sudah ditinggalkan bisa memuaskan request yang masih hidup.

Dua percobaan dari satu request klien adalah dua percakapan backend yang berbeda, dengan identifier berbeda. Ini yang membuat jawaban terlambat dari percobaan pertama tidak mungkin dipasangkan ke percobaan kedua.

Per transport

HTTP. `http.Transport` standar dengan connection pool. Korelasi lewat header `X-Request-ID` dan `request_id` di body respons, keduanya diperiksa.

Frame TCP. Satu writer mutex, satu goroutine pembaca, dan `map[uint64]chan` pending per koneksi. Inilah hasil langsung dari temuan di bagian 3. Respons dipasangkan berdasarkan identifier, tidak pernah berdasarkan urutan kedatangan. Pool-nya kecil dengan pemilihan koneksi least-loaded, dan dial saat cold start diserialisasi supaya lonjakan request di awal tidak membuka soket yang langsung dibuang.

Datagram UDP. Satu socket per backend dengan satu goroutine demux. Satu socket per request akan membakar satu ephemeral port setiap panggilan dan kehilangan estimasi RTT bersama.

Backpressure

Ada tiga tingkat, dan semuanya menolak cepat alih-alih mengantre tanpa batas:

Tingkat	Default	Perilaku saat penuh
Admission global	512	<code>GATEWAY_OVERLOADED</code> , retryable
Per backend	128	<code>SERVICE_UNAVAILABLE</code> , retryable, aman fallback
Window UDP in-flight	128	<code>BACKEND_UNAVAILABLE</code> , belum terkirim, aman dialihkan

Menolak cepat lebih jujur daripada mengantre. Caller memegang deadline-nya sendiri, dan pekerjaan yang tidak akan selesai dalam budget itu tidak berguna bagi siapa pun.

Budget waktu

Satu deadline dibentuk dari `timeout_ms` caller, dikurangi margin respons 120 ms supaya jawaban gateway mendahului deadline caller sendiri. Setiap percobaan mengiris dari sisa budget itu, dibatasi `BABEL_ATTEMPT_MAX_MS` . Backoff antar percobaan bersifat deterministik, diturunkan dari indeks percobaan tanpa randomisasi, supaya rangkaian kegagalan bisa diputar ulang persis sama.

11. Penanganan kegagalan

Model kegagalan

Setiap kegagalan diklasifikasikan pada dua sumbu yang saling bebas:

- **Retryable**, yaitu apakah mencoba lagi mungkin berhasil;
- **FallbackSafe**, yaitu klaim yang jauh lebih kuat bahwa gateway tahu percobaan yang gagal tidak menghasilkan efek apa pun yang terlihat backend.

Perbedaan dua sumbu inilah yang menentukan apakah gateway boleh mengalihkan pekerjaan ke backend lain. Isinya diturunkan langsung dari pemetaan ledger di bagian 5.

Kegagalan	Retryable	FallbackSafe	Alasan
Koneksi ditolak atau gagal dial	ya	ya	Tidak ada byte yang sampai ke wire
Koneksi diputus tanpa respons	ya	ya	Ledger membuktikan tidak sampai tahap eksekusi
HTTP 503 atau 429	ya	ya	Backend menolak sebelum menjalankan
Breaker menolak	ya	ya	Ditolak sebelum satu byte pun dikirim
Error domain dari backend	dari backend	ya	Penolakan terstruktur berarti tidak dieksekusi
Payload terlalu besar	tidak	ya	Ditolak sebelum transmisi
Timeout	ya	tidak	Backend mungkin masih mengerjakannya
Respons rusak atau checksum salah	ya	tidak, lihat catatan	Backend sudah selesai, tapi hasilnya tak terpakai
Versi protokol tidak didukung	tidak	tidak, lihat catatan	Sama seperti di atas, backend sudah menjawab

Baris terakhir sempat saya salah klasifikasikan. Awalnya saya menandai ketidaksesuaian versi sebagai fallback-safe dengan alasan "ini ketidaksepakatan statis, backend lain bisa saja cocok". Alasan itu benar untuk sumbu **Retryable**, tetapi salah untuk sumbu **FallbackSafe**. Ledger menunjukkan bahwa di bawah fault `unsupported_version`, Service A tetap mencapai tahap `executed`. Ia menjawab, hanya saja menjawab dengan versi yang tidak bisa gateway pahami. Jadi tempatnya bukan di kelompok "terbukti tidak bekerja", melainkan di kelompok "sudah selesai tetapi jawabannya tidak terpakai", bersama respons rusak. Saya perbaiki klasifikasinya agar konsisten dengan bukti, bukan dengan intuisi awal saya.

Gerbang fallback

```
boleh fallback jika FallbackSafe
                  atau (jawaban tidak terpakai dan BABEL_FALLBACK_ON_CORRUPT aktif)
                  atau (Retryable dan operasi ditandai replay_safe)
```

Kategori "jawaban tidak terpakai" mencakup payload yang tidak bisa di-parse, checksum yang gagal, identifier yang tidak cocok, dan versi protokol yang tidak dipahami adapter.

Timeout tidak pernah memicu fallback secara default. Ini bukan sikap konservatif yang asal-asalan. Bukti empirisnya ada di bagian 5: fault `delayed_response` tetap mengeksekusi operasinya setelah delay. Gateway yang timeout lalu berpindah backend akan menghasilkan dua entri `executed` untuk satu request logis, dan skenario `fallback-unsafe` memang menginjeksi fault ini.

Sebaliknya, di bawah `fallback-safe` dan `partial-outage`, ledger hanya mencatat `received` tanpa `executed`. Fallback di sana menghasilkan tepat satu eksekusi, dan itulah yang gateway lakukan.

Respons rusak adalah kasus tengah yang saya putuskan dengan jujur. Respons yang tidak bisa dipercaya sudah membuat request gagal apa pun yang terjadi. Pilihannya cuma dua: memberi caller tidak apa-apa, atau mencoba backend lain dengan risiko operasinya berjalan dua kali dalam kasus terburuk. Saya memilih mencoba, karena seluruh operasi di deployment ini adalah fungsi murni tanpa efek samping, sehingga eksekusi kedua tidak mengubah state apa pun, sementara alternatifnya adalah kegagalan yang dijamin.

Ini klaim yang lebih lemah daripada baris pertama tabel, dan saya sengaja memisahkannya menjadi satu flag tersendiri. `BABEL_FALLBACK_ON_CORRUPT=false` mengambil pembacaan paling ketat.

`replay_safe` adalah pernyataan operator per pasangan (service, operasi), default `false`, dan bisa diubah saat runtime. Semantiknya lebih kuat daripada sekadar "idempotent". Kelima operasi di sini memang murni, tetapi environment mencatat setiap eksekusi di ledger yang terlihat dari luar, jadi eksekusi kedua tetap teramati meskipun tidak berbahaya secara semantik.

Isolasi kegagalan

Circuit breaker per pasangan (service, versi protokol), bukan per service. Saat rolling upgrade, v1 bisa sehat sementara v2 rusak. Menyatukan breaker-nya akan menyembunyikan versi yang buruk atau justru menghukum yang baik. Demo bonus bagian B menunjukkan ini terjadi sungguhan: `service-b#v2` terbuka sementara `service-b#v1` tetap tertutup dan melayani.

Error domain tidak menaikkan hitungan breaker. Backend yang dengan benar menolak argumen buruk sedang bekerja dengan sempurna. Menghitungnya akan membuat breaker terbuka karena kesalahan klien dan mengeluarkan backend sehat dari rotasi.

Kegagalan transport ditangani bergradasi, karena katalog fault memang membedakannya:

- frame yang tidak dinanti siapa pun, termasuk duplikat, dihitung lalu dibuang. Duplicate suppression jadi didapat cuma-cuma, karena respons pertama sudah menghapus penunggunya dari peta pending;

- frame dengan magic atau versi salah tetapi panjang masuk akal berarti stream masih sinkron, jadi hanya request yang bersangkutan yang gagal;
- panjang yang dilarang protokol berarti posisi stream tidak diketahui. Tidak ada cara aman mencari batas frame berikutnya, jadi koneksi dirobuhkan. Koneksi lain dan seluruh backend lain tidak tersentuh;
- header datang tetapi payload tidak pernah lengkap ditangani deadline body frame selama 1500 ms, yang mengubahnya menjadi pelanggaran protokol alih-alih timeout yang menghabiskan seluruh budget caller.

Panic tidak pernah lolos. Handler HTTP, probe health, dan konstruksi adapter semuanya berjalan di bawah `recover`. Sebuah panic menjadi HTTP 500 dengan body JSON yang valid, dan stack trace-nya masuk ke log tempat ia seharusnya berada.

Health checking

Sesuai temuan di bagian 5, probe hanya diperlakukan sebagai sinyal liveness. Status yang dilaporkan menggabungkan probe dengan hasil trafik nyata:

probe gagal	-> unavailable
probe lolos, breaker terbuka	-> degraded (trafik nyata tidak setuju)
probe lolos, breaker tertutup	-> available
tidak ada probe yang mungkin	-> diturunkan dari breaker saja

Backend datagram tidak punya handshake untuk diamati, jadi satu-satunya bukti adalah operasi yang terjawab, dan operasi itu dicatat backend di ledger persis seperti trafik klien. Probe semacam ini saya tandai intrusif dan jalankan pada irama lambat 30 detik, naik ke irama normal hanya ketika backend sudah terlihat tidak sehat. Gateway tidak boleh terus-menerus memproduksi pekerjaan yang tidak diminta siapa pun.

12. Bonus 1: migrasi protokol saat runtime dan kompatibilitas

Dua bonus ini saya kerjakan sebagai satu mekanisme, karena memang keduanya adalah hal yang sama dilihat dari sudut berbeda.

Sebuah service memegang beberapa **variant protokol berbobot** sekaligus:

- rolling upgrade adalah `{v1: 70, v2: 30}`
- cutover adalah `{v1: 0, v2: 100}`
- rollback adalah kebalikannya

Pemilihan versi memakai hash FNV-1a dari `request_id` digabung nama operasi, modulo total bobot. Sifatnya deterministik, jadi request yang sama selalu mendarat di versi yang sama. Ini yang membuat canary bisa dipikirkan ulang alih-alih menjadi lemparan koin, dan membuat retry tidak diam-diam

berpindah protokol di tengah insiden.

Endpoint `POST /admin/migrate` menjalankan tiga langkah, dan urutannya menjaga setiap keadaan antara tetap aman:

1. adapter versi target dibangun dan dibuktikan dengan vektornya sendiri;
2. variant didaftarkan, selalu pada bobot nol lebih dulu, sehingga mendaftarkan sebuah versi tidak pernah dengan sendirinya memindahkan trafik;
3. bobot digeser.

Gagal di langkah mana pun meninggalkan langkah sebelumnya utuh dan tetap melayani. Setiap langkah dipersistensi begitu diterapkan, jadi migrasi bertahan melewati restart.

Yang juga penting: **request yang sedang berjalan selesai pada instance adapter tempat mereka mulai**. Manager menghitung referensi. Sebuah swap memasang instance baru untuk akuisisi berikutnya lalu mendrain yang lama, dan baru menutupnya setelah penunggu terakhir melepaskannya atau deadline drain habis.

Demo bonus bagian B melakukan canary ke versi 2 yang sengaja tidak dipahami backend referensi. Kegagalan yang muncul justru intinya: rollout yang buruk harus gagal dengan aman, terisolasi ke breaker-nya sendiri, dan bisa dikembalikan.

13. Bonus 2: pemuatan adapter saat runtime

`POST /admin/adapters` memuat atau mengganti adapter tanpa restart. Urutan operasinya adalah keseluruhan cerita keamanannya:

1. spec divalidasi secara struktural;
2. instance dibangun di bawah `recover` dan sebuah deadline, karena spec yang rusak atau jahat tidak boleh menggantung atau menjatuhkan gateway;
3. instance harus mereproduksi golden vector-nya, persis;
4. baru dipublikasikan, dan yang lama di-drain, bukan ditutup mendadak;
5. spec dipersistensi.

Gagal di langkah mana pun mengembalikan error dan tidak mengubah apa pun. Adapter sebelumnya tetap melayani.

Demo bonus bagian A menunjukkan ini di bawah trafik langsung. Ada 87 request selama swap berlangsung, nol di antaranya gagal. Lalu spec yang rusak dicoba dimuat, ditolak, dan generasi adapter yang sedang berjalan tidak berubah sama sekali.

14. Bonus 3: integrasi lanjutan

Di atas datagram yang tidak menjanjikan apa pun, paket `transport/udpre1` menambahkan:

- **korelasi** pada pasangan `(request id, sequence)`, tidak pernah pada urutan kedatangan;
- **duplicate suppression**, termasuk untuk salinan yang tiba setelah jawaban sudah dikirim ke pemanggil, lewat memori identifier selesai yang terbatas ukurannya dan punya TTL;
- **validasi integritas sebelum payload disentuh**. Magic, versi, dan panjang yang dideklarasikan dicocokkan dengan ukuran datagram nyata sebelum CRC dihitung, supaya field panjang yang jahat tidak bisa membuat pembacaan di luar batas array;
- **retransmisi adaptif** dengan estimator Jacobson dan Karels sesuai RFC 6298, ditambah algoritma Karn. Sampel RTT hanya diambil dari transmisi yang belum pernah diulang, karena sampel dari salinan lama akan meracuni estimatornya;
- **exponential backoff** pada kehilangan berulang, dibatasi `MaxRTO`;
- **window in-flight terbatas** sebagai sinyal backpressure;
- **reassembly fragmen** di jalur terima, toleran terhadap kedatangan yang tidak berurutan.

Ada satu keputusan yang layak disorot: **datagram yang rusak di-drop, bukan dijadikan error**. Pada tautan yang lossy, datagram rusak tidak bisa dibedakan dari datagram yang tidak pernah tiba, jadi satu-satunya tafsir yang aman adalah menganggapnya tidak pernah sampai, dan membiarkan timer retransmisi yang menanganinya. Inilah sebabnya skenario `service-c-corrupt-checksum` dan `service-c-lossy` berhasil delapan dari delapan pada demo, bukan gagal.

Sisi TCP menyumbang multiplexing dengan korelasi tanpa urutan, batas in-flight per koneksi, pemilihan koneksi least-loaded, dan deadline body frame.

Fragmentasi sisi kirim saya matikan untuk backend referensi, yang akan menolak request terfragmentasi. Payload yang berlebih ditolak sebelum transmisi dengan kode `PAYLOAD_TOO_LARGE` yang bersifat fallback-safe, sehingga router bisa memilih backend yang berorientasi stream. Jalur terima tetap melakukan reassembly dan diuji terhadap server loopback.

15. Bonus 4: penanganan di luar spesifikasi

Beberapa hal saya tangani karena environment atau kewarasan menuntutnya, bukan karena diminta soal.

Registry rusak dikarantina, tidak ditimpa, dan peringatannya muncul di `/status`.

Schema version yang tidak dikenal ditolak keras, karena menebak isi dokumen dari build masa depan lebih berbahaya daripada menolaknya.

Correlation id aman melintasi restart, termasuk saat jam sistem mundur.

Reuse identifier yang masih in-flight ditolak di lapisan transport. Kalau suatu saat alokator bocor, ia akan berbunyi keras, bukan diam-diam memasang dua request.

Deadline body frame mengubah fault `truncated_frame` dari penghabis budget menjadi pelanggaran protokol yang terdeteksi cepat.

Dial cold-start diserialisasi. Ini ditemukan lewat test yang mendeteksi pool membuka lebih banyak socket daripada ukuran yang dikonfigurasi ketika banyak request datang bersamaan di awal.

Presisi spec dipersistensi byte demi byte. Ini bug paling menarik yang saya temukan. Adapter yang di-hot-load gagal self-test-nya sendiri setelah restart, padahal tidak ada yang berubah padanya. Penyebabnya: spec yang dipersistensi di-decode lewat `map[string]any`, dan Go mengubah setiap angka JSON menjadi `float64`. Correlation id `1234605616436508552` di dalam golden vector kembali sebagai `1234605616436508700`. Presisi 64-bit hilang diam-diam. Sekarang spec disimpan sebagai `json.RawMessage` dan ada regression test-nya. Yang menangkap bug ini bukan review manual, melainkan golden vector yang tiba-tiba tidak cocok.

Invariant envelope ditegakkan sekali lagi tepat sebelum byte meninggalkan proses. Pelanggaran menjadi internal error yang ter-log, bukan envelope cacat di wire.

Shutdown menepati janji yang sudah dibuat. Berhenti menerima request baru, biarkan yang sedang berjalan selesai, persistensi high water, baru tutup adapter.

Probe intrusive dibatasi iramanya supaya health checking tidak memproduksi trafik backend terus-menerus.

16. Keputusan teknis dan kompromi

Status HTTP: 200 dengan envelope error

Ini kompromi yang paling perlu dijelaskan. Kontrak Gateway API mencantumkan status untuk kondisi level gateway, misalnya 408 saat timeout dan 503 saat tidak ada rute. Tetapi kontrak yang sama juga menyatakan bahwa envelope adalah yang dinilai. Dan ketika saya membaca klien referensi di `towerofbabel/client/gateway_connector.py`, ternyata ia memanggil `response.raise_for_status()`.

Artinya balasan non-2xx apa pun akan menggagalkan klien sebelum envelope sempat dibaca. Menjawab 408 untuk sebuah timeout berarti mengganti envelope error yang presisi dan terbaca mesin dengan sebuah exception di sisi klien.

Jadi gateway ini menjawab 200 dengan envelope error yang lengkap untuk setiap hasil level domain, dan menyisakan non-2xx untuk pelanggaran kontrak caller (400) dan cacat gateway sendiri (500). Kompromi ini tidak saya sembunyikan: `BABEL_STRICT_HTTP_STATUS=true` memilih pemetaan literal, dan envelope-nya identik byte per byte di kedua mode.

Tidak ada hedging

Sudah dijelaskan di bagian 10. Hedging memperbaiki latensi ekor tetapi langsung melanggar larangan operasi duplikat.

Retry pada service yang sama sebelum berpindah

Sampai dua percobaan pada satu backend sebelum berpindah, dan hanya untuk kegagalan yang fallback-safe. Fault sekali tembak seperti `connection_termination` pulih di percobaan kedua tanpa perlu melibatkan backend lain. Lebih sedikit lompatan, dan ledger tetap menunjukkan satu eksekusi.

Error backend diteruskan apa adanya

Tidak dipetakan ke kosakata gateway. Membuat pemetaan berarti membuang informasi yang dimiliki backend dan tidak dimiliki gateway.

Probe metadata untuk Service C

Socket datagram tidak menawarkan handshake, jadi satu-satunya cara mengetahui backend menjawab adalah menanyakan sesuatu kepadanya. Kompromi yang saya akui: probe itu masuk ke ledger eksekusi backend. Mitigasinya adalah irama yang lambat dan eskalasi hanya saat backend terlihat tidak sehat. Alternatifnya, yaitu melaporkan Service C sebagai `unknown` sampai trafik nyata datang, saya nilai lebih buruk untuk requirement observability.

Admin API terbuka secara default

Terbuka di dalam jaringan compose, dan portnya tidak dipublikasikan keluar. Ini default yang keliru untuk produksi. `BABEL_ADMIN_TOKEN` menutupnya dan `BABEL_ADMIN_ENABLED=false` menghapus route-nya sama sekali.

Go dengan standard library saja

Kontrol byte yang eksplisit untuk protokol biner, primitif konkurensi yang ringan untuk tiga model transport berbeda, binary statis, dan build yang hermetik tanpa perlu mengunduh apa pun.

17. Keterbatasan implementasi

Bagian ini berisi hal-hal yang benar-benar keterbatasan, bukan keputusan.

Transport family baru tetap butuh kode Go. Spec mengomposisi tiga family yang sudah ada. gRPC, WebSocket, atau protokol lain memerlukan implementasi family baru. Bagian deklaratifnya menutupi apa yang biasanya berubah pada revisi protokol, yaitu nama field, token operasi, header, dan batas payload, bukan mekanisme transportnya sendiri.

Retransmisi datagram bersifat at-least-once. Kalau yang hilang adalah responsnya, backend sudah mengeksekusi dan akan mengeksekusi lagi. Ini melekat pada ARQ dan tidak bisa dihindari. Itulah sebabnya router tidak pernah menambahkan fallback lintas backend di atas timeout UDP, karena melakukan keduanya akan melipatgandakan duplikasi alih-alih menahannya.

Fragmentasi sisi kirim tidak aktif terhadap backend referensi. Jalur terimanya lengkap dan teruji, tetapi tidak pernah dilatih oleh backend nyata karena backend itu tidak pernah memfragmentasi.

Migrasi hanya lintas versi dalam family yang sama. Memindahkan sebuah service dari, misalnya, framed TCP ke HTTP memerlukan definisi service baru, bukan sekadar variant baru.

`/status` tidak diautentikasi dan memaparkan endpoint serta counter.

Health tidak dipersistensi setiap probe, jadi setelah restart nilai `last_known_health` bisa basi satu interval sampai sweep pertama selesai.

Tidak ada distributed state. Beberapa instance gateway akan punya registry, breaker, dan estimasi RTT masing-masing. Cukup untuk deployment ini, tetapi koordinasi lintas instance adalah pekerjaan yang berbeda.

Backoff deterministik tanpa jitter. Saya pilih demi reproduksibilitas saat penilaian. Pada armada besar, backoff tanpa jitter justru menyinkronkan retry, dan produksi menginginkan jitter di sini.

18. Verifikasi

Setiap klaim di laporan ini saya cek terhadap stack yang berjalan, bukan disimpulkan dari kode.

Bukti	Cakupan
<code>go test -race</code> <code>./...</code>	149 test: framing dan codec, golden vector, dedupe dan reassembly UDP, multiplexing dan isolasi kegagalan TCP, keunikan identifier lintas restart, persistensi registry, gerbang fallback router, invarian envelope
Test dijalankan di dalam Docker build	Image yang berhasil dibangun adalah image yang test-nya lulus
Sweep fungsional	17 skenario fault publik lewat gateway, setiap respons dicek terhadap kontrak envelope, hasilnya 37 dari 37 lulus terhadap stack compose
Ledger control plane	Klaim tanpa duplikasi diperiksa terhadap catatan eksekusi backend, bukan terhadap log gateway sendiri
Klien referensi	Skenario <code>full-demo</code> dan <code>concurrency</code> bawaan kit dijalankan dan lulus
<code>./demo/run-all.sh</code>	Sembilan poin demonstrasi wajib ditambah setiap bonus, end to end

Dua defect ditemukan lewat verifikasi ini dan diperbaiki, bukan ditutupi. Pertama, connection pool bisa melampaui ukurannya saat cold start yang konkuren. Kedua, spec adapter yang dipersistensi kehilangan presisi integer 64-bit. Keduanya sekarang punya regression test yang gagal kalau bug-nya kembali.

Cara menjalankan ulang semuanya:

```
docker compose up -d  
./demo/run-all.sh  
cd src && go test -race ./...
```